

Integrazione LLM nel Data Warehouse Olist

Documentazione Tecnica degli Script

10_script_llm

Emanuele Colecchia — Matricola 276527

Corso: Data Warehouse and Visualization

A.A. 2025/2026 — Università della Calabria

Dataset: Brazilian E-Commerce Public Dataset (Olist, Kaggle)

Contents

1 Introduzione Teorica: LLM come Livello di Supporto

1.1 Ruolo degli LLM nel processo DQ

I Large Language Models (LLM) vengono utilizzati in questi script come **livello di supporto** aggiuntivo rispetto alla pipeline classica di Data Quality Assessment (DQA). Il principio fondamentale è che *gli LLM non sostituiscono la pipeline quantitativa*: gli scorecard DQ rimangono la fonte autoritativa di verità, mentre gli LLM arricchiscono il processo con:

- **Interpretazione in linguaggio naturale** degli scorecard tecnici, traducendoli in report leggibili da stakeholder non tecnici.
- **Generazione di regole di validità domain-aware**: dato il contesto reale del dominio (e-commerce brasiliano, categorie prodotti, stati geografici), l'LLM suggerisce lambda Python specifiche per il dataset.
- **Classificazione della missingness**: supporto alla distinzione tra MCAR / MAR / MNAR con ragionamento domain-aware.
- **Pianificazione strutturata del cleaning**: generazione di piani JSON con priorità e metodi pandas specifici.
- **Narrazione degli audit log**: trasformazione di log tecnici in report comprensibili per il management.

1.2 Provider LLM supportati

Tutti e tre gli script supportano i seguenti provider, configurabili tramite la variabile LLM_PROVIDER:

Provider	Modello	Requisito	Note
openai	gpt-4o-mini	API key OpenAI	Cloud
anthropic	claude-haiku-4-5-20251001	API key Anthropic	Cloud
ollama	llama3.2:3b	Server locale	Default
huggingface	Qwen/Qwen2.5-1.5B-Instruct	Token HF (opt.)	Cloud

Table 1: Provider LLM supportati dalla pipeline

1.3 Architettura dei path

Gli script sono stati adattati dalla versione originale (Google Colab + Google Drive) a un'architettura **portable e locale**. I path vengono calcolati automaticamente a partire dalla posizione dello script:

```
_HERE = os.path.dirname(os.path.abspath('__file__')) #
consegna/10_script_llm/
```

```
_ROOT = os.path.dirname(_HERE) #
consegna/
RAW_PATH = os.path.join(_ROOT, '1_raw_data') + os.sep
CLEANED_PATH = os.path.join(_ROOT, '3_cleaned_data') + os.sep
OUTPUT_PATH = os.path.join(_ROOT, '10_script_llm', 'outputs') +
os.sep
```

Questo garantisce che gli script funzionino su qualsiasi macchina senza modifiche manuali ai path.

2 Modifiche Apportate — Adattamento al Dataset Olist

2.1 Problemi nella versione originale

La versione originale dei notebook presentava i seguenti problemi che impedivano l'esecuzione:

1. **Path Google Colab non portabili:** `BASE_PATH = '/content/drive/Shareddrives/...'` e `drive.mount('/content/drive')` — inutilizzabili al di fuori di Colab.
2. **Dataset generici non correlati a Olist:**
 - L1: `orders_table.csv` (dati sintetici generici)
 - L2: `patient_readings.csv` (dati medici, nessuna relazione con il progetto)
 - L3: `L3_products_dirty.csv` (dati retail generici)
3. **Prompt LLM con contesto sbagliato:** riferimenti a dataset medici, colonne inesistenti (`admission_date`, `category`, `brand`).
4. **Celle di installazione sistema inutili:** `!apt-get install zstd, !curl -fsSL https://ollama.com/install.sh | sh` — specifiche di Colab.
5. **Codice duplicato:** funzione `call_llm_ollama` definita più volte, poi ridefinita come `call_llm` — confusione e ridondanza.

2.2 Interventi effettuati

Problema	Versione originale	Versione adattata
Path	/content/drive/SharedDrivePath	Path relativi con os.path
Dataset L1	orders_table.csv (sintetico)	olist_orders_dataset.csv
Dataset L2	patient_readings.csv (medico)	olist_orders_dataset.csv +
Dataset L3	L3_products_dirty.csv (generico)	olist_order_items_dataset.csv olist_products_dataset.csv + dim_products.csv
Scorecard	File generici	4_dq_scorecards/dq_scorecard_*.csv reali
Colab setup	drive.mount(), !apt-get	Rimossi
Prompt LLM	Contesto generico/medico	Contesto Olist (BRL, 2016-2018, BR)
Funzione LLM	Duplicata e incompleta	Unificata in call_llm() completa

Table 2: Modifiche apportate per adattare i notebook al dataset Olist

3 L1 — LLM-Supported Data Quality Assessment

3.1 Descrizione

Lo script L1 utilizza gli LLM per analizzare gli scorecard DQ già prodotti dalla pipeline classica (`data_quality_assessment_LOCAL.ipynb`) e produrre tre output in linguaggio naturale:

1. **Audit Report:** interpretazione professionale dello scorecard per stakeholder.
2. **Regole di validità:** generazione di lambda Python domain-aware.
3. **Sommario stakeholder:** spiegazione semplificata per management non tecnico.

3.2 Input

- 1_raw_data/olist_orders_dataset.csv (99.441 righe)
- 1_raw_data/olist_customers_dataset.csv (99.441 righe)
- 1_raw_data/olist_order_items_dataset.csv (112.650 righe)
- 1_raw_data/olist_products_dataset.csv (32.951 righe)
- 1_raw_data/olist_sellers_dataset.csv (3.095 righe)

- 1_raw_data/olist_order_payments_dataset.csv (103.886 righe)
- 1_raw_data/olist_order_reviews_dataset.csv (99.224 righe)
- 1_raw_data/olist_geolocation_dataset.csv (1.000.163 righe)
- 4_dq_scorecards/dq_scorecard_*.csv (scorecard per ogni tabella)

3.3 Output

- 10_script_llm/outputs/dq_audit_{table}.md — report audit per ogni tabella
- 10_script_llm/outputs/dq_generated_rules_{table}.md — regole validità
- 10_script_llm/outputs/stakeholder_summaries.md — sommari stakeholder

3.4 Prompt LLM — Use Case 1: Audit Report

Listing 1: System prompt per audit report

```
SYSTEM_AUDIT = """You are a senior Data Quality Analyst
specializing in Data Warehouse
systems for large-scale e-commerce platforms. The dataset is from
Olist, the largest
Brazilian online marketplace. Your task is to interpret a
structured DQ scorecard and
produce a clear, actionable audit report for a business
stakeholder audience.
Be specific about root causes and prioritize recommendations by
business impact.
Write in Italian."""
```

Listing 2: User prompt per audit report (template)

```
PROMPT_AUDIT = f"""Di seguito e' riportato lo scorecard DQ per la
tabella '{table_name}'
del Data Warehouse Olist (e-commerce brasiliano, ~100.000 ordini
reali, 2016-2018).
Analizza e produci un report strutturato con:
1. Riepilogo Esecutivo (2-3 frasi)
2. Problemi Critici (richiedono intervento immediato)
3. Problemi Moderati (da risolvere prima del prossimo ciclo ETL)
4. Priorita' di Cleaning Raccomandata (lista ordinata per impatto
)
5. Rischio Stimato per Analytics Downstream

SCORECARD:
{scorecard_json}
"""
```

3.5 Prompt LLM — Use Case 2: Generazione Regole di Validità

Listing 3: System prompt per generazione regole

```
SYSTEM_RULES = """You are a Data Engineer expert in Data Quality
for e-commerce DW systems.
Given column metadata and sample values from the Olist Brazilian
e-commerce platform,
generate Python pandas validation rule functions (lambdas) using
only standard
pandas/numpy operations.
Each rule must be a lambda that takes a Series and returns a
boolean Series
(True = valid).
Return ONLY valid Python code in a dict named 'validity_rules'.
Use domain knowledge: Brazilian state codes (2 uppercase letters)
, UUID-format IDs,
positive prices, order_status in a defined set, etc."""
```

Listing 4: Esempio output atteso dalle regole generate

```
validity_rules = {
    'order_id': lambda s: s.str.len() == 32,
    'customer_id': lambda s: s.str.len() == 32,
    'order_status': lambda s: s.isin(['delivered', 'shipped', '
        processing',
                                     'canceled', 'unavailable',
                                     'invoiced',
                                     'created', 'approved']),
    'order_purchase_timestamp': lambda s: pd.to_datetime(s,
        errors='coerce').notna(),
}
```

4 L2 — LLM-Supported Missing Values & Outlier Analysis

4.1 Descrizione

Lo script L2 analizza i pattern di valori mancanti e gli outlier nei dataset Olist, con supporto LLM per:

1. **Interpretazione MCAR/MAR/MNAR:** classificazione domain-aware della tipologia di missingness.
2. **Strategia di imputazione:** generazione di codice Python per la gestione ottimale dei NULL.
3. **Narrativa outlier:** spiegazione business degli outlier rilevati.

4.2 Dataset analizzati

- `olist_orders_dataset.csv`: missingness su `order_approved_at` (160 NULL), `order_delivered_customer_date` (1.783 NULL), `order_delivered_customer_date` (2.965 NULL).
- `olist_order_items_dataset.csv`: outlier su `price` (max 6.735 BRL) e `freight_value`.
- `olist_products_dataset.csv`: outlier su misure fisiche (es. peso 40.000g), missingness su colonne dimensionali.
- `olist_customers_dataset.csv`: analisi completezza e unicità.
- `olist_sellers_dataset.csv`: analisi distribuzione geografica e outlier.
- `olist_order_payments_dataset.csv`: outlier su `payment_value` e `payment_installments`.
- `olist_order_reviews_dataset.csv`: missingness su `review_comment_title` e `review_comment`.
- `olist_geolocation_dataset.csv`: outlier su coordinate lat/lng.

4.3 Classificazione Missingness

La classe `MissingnessAnalyzer` applica un'euristica basata su:

- **MCAR**: percentuale di missingness $< 0.5\%$ — probabile casualità.
- **MAR**: missingness correlata a variabili categoriche osservabili (variazione $> 5\%$ del tasso di missingness tra categorie).
- **MNAR**: nessuna correlazione rilevata — possibile null semantico (es. ordine non ancora consegnato).

Per il dataset Olist, i tre campi mancanti sono classificati come **MNAR (null semantici)**: la mancanza di `order_delivered_customer_date` non è un errore — significa semplicemente che l'ordine non è ancora stato consegnato.

4.4 Prompt LLM — Interpretazione Missingness

Listing 5: System prompt per interpretazione missingness

```
SYSTEM_MISSING = """You are a senior Data Engineer specializing
in e-commerce
Data Warehouses. You are analyzing the Olist dataset, a Brazilian
e-commerce
platform with ~100,000 orders from 2016-2018.
Your role is to interpret missing value patterns, classify them
as MCAR/MAR/MNAR
with domain knowledge, and suggest appropriate imputation or
handling strategies.
Write in Italian. Be specific and use domain context (e.g.,
orders without
```



```
delivery dates may simply not have been delivered yet -- semantic
nulls,
not data errors)."""
```

Listing 6: User prompt per interpretazione missingness (estratto)

```
PROMPT_MISSING = f"""Tabella: olist_orders (99.441 ordini e-
commerce brasiliani, 2016-2018)
Colonne con valori mancanti:
{miss_json}

Nota di dominio:
- order_approved_at: data di approvazione pagamento. NULL se il
pagamento
non e' stato ancora approvato.
- order_delivered_carrier_date: data di consegna al corriere.
NULL se
l'ordine e' ancora in lavorazione.
- order_delivered_customer_date: data di consegna al cliente.
NULL se
non ancora consegnato.

Per ogni colonna:
1. Classifica il tipo di missingness (MCAR/MAR/MNAR) con
motivazione
2. Spiega l'impatto sul DW se i NULL non vengono gestiti
correttamente
3. Raccomanda la strategia ottimale (flag binario, esclusione,
imputazione)
"""
```

4.5 Outlier Detection — Metodi

Metodo	Formula	Uso
IQR ($k = 1.5$)	$[Q_1 - 1.5 \cdot IQR, Q_3 + 1.5 \cdot IQR]$	Default
Z-score ($\sigma = 3$)	$ z > 3$ dove $z = (x - \mu)/\sigma$	Distribuz. normali

Table 3: Metodi di outlier detection implementati in OutlierDetector

4.6 Prompt LLM — Narrativa Outlier

Listing 7: System prompt per narrativa outlier

```
SYSTEM_OUTLIER = """You are a senior Data Analyst for Olist, a
Brazilian
e-commerce marketplace. You interpret outlier detection results
and provide
```

```
business-context narrative explanations. For each detected
outlier:
- Explain potential real-world causes (luxury products, errors,
  fraud, etc.)
- Assess business impact (distorted averages, wrong
  recommendations, etc.)
- Recommend the appropriate handling strategy
Write in Italian."""
```

5 L3 — LLM-Supported Data Cleaning

5.1 Descrizione

Lo script L3 utilizza gli LLM per supportare il processo di cleaning di tutte le 8 tabelle Olist, con due use case principali applicati sistematicamente:

1. **Piano di cleaning strutturato:** l'LLM genera per ogni tabella un piano JSON con step numerati, metodo pandas da applicare, priorità (high/medium/low) e impatto sul business se lo step non viene eseguito.
2. **Narrativa audit:** dato il confronto statistico prima/dopo cleaning, l'LLM produce una narrativa professionale delle trasformazioni effettuate, quantificando il miglioramento della qualità.

Per la tabella `products`, è presente anche un terzo use case specifico: mapping delle categorie dal portoghese all'inglese, con verifica del mapping ufficiale Olist e integrazione LLM per le categorie non coperte.

5.2 Input/Output

File	Tipo	Contenuto
<code>olist_products_dataset.csv</code>	Input raw	32.951 prodotti sporchi
<code>product_category_name_translation.csv</code>	Reference	71 mapping PT→EN
<code>dim_products.csv</code>	Input cleaned	Versione pulita post-pipeline
<code>audit_log.csv</code>	Input	Log trasformazioni
<code>L3_category_mapping.json</code>	Output	Mapping finale PT→EN
<code>L3_cleaning_plan.json</code>	Output	Piano cleaning strutturato
<code>L3_audit_narrative.md</code>	Output	Narrativa audit in italiano

Table 4: File di input e output di L3

5.3 Prompt LLM — Use Case 1: Mapping Categorie

Listing 8: System prompt per mapping categorie PT→EN

```
SYSTEM_ENUM = """You are a Data Engineer building a product
taxonomy for a
Brazilian e-commerce DW. The platform is Olist, an online
marketplace
aggregating multiple sellers.
You receive Portuguese product category names and must map them
to standardized
English equivalents.
Rules:
- Use standard e-commerce taxonomy (Electronics, Home & Garden,
Sports,
Beauty & Health, etc.)
- Return ONLY a Python dict named 'category_mapping' with '
pt_name':'en_name'
- If a category is 'nan' or invalid, map it to 'unknown'
- Be consistent: same concept -> same English label"""
```

Il mapping ufficiale contiene 71 coppie PT→EN. Il sistema usa `product_category_name_translation.csv` come ground truth e invoca l'LLM solo per categorie eventualmente non mappate nel file ufficiale.

5.4 Prompt LLM — Use Case 2: Piano di Cleaning

Listing 9: System prompt per piano cleaning

```
SYSTEM_PLAN = """You are a senior Data Engineer building ETL
cleaning pipelines
for a Brazilian e-commerce Data Warehouse (Olist). You produce
structured,
actionable cleaning plans in JSON format that can be directly
translated into
Python/pandas code. Prioritize by business impact. Be specific
about the
pandas method to use. Return ONLY valid JSON."""
```

Listing 10: Struttura JSON attesa dal piano cleaning

```
{
  "table": "olist_products_dataset",
  "steps": [
    {
      "step_id": 1,
      "name": "fill_missing_category",
      "description": "Imputa le categorie mancanti con 'unknown'",
      "columns": ["product_category_name"],
      "pandas_method": "fillna('unknown')",
      "priority": "high",
      "business_impact": "Categorie NULL rendono impossibile l'
```

```

        analisi fatturato per categoria"
    },
    {
        "step_id": 2,
        "name": "add_english_category",
        "description": "Aggiunge colonna
            product_category_name_english tramite JOIN",
        "columns": ["product_category_name"],
        "pandas_method": "map(cat_translation_dict)",
        "priority": "high",
        "business_impact": "Dashboard Tableau richiede categorie in
            inglese"
    }
]
}

```

6 Istruzioni per l'Esecuzione

6.1 Prerequisiti

Listing 11: Installazione dipendenze Python

```
pip install openai anthropic scikit-learn scipy requests --quiet
```

Per usare **Ollama** (provider consigliato, locale, gratuito):

Listing 12: Installazione e avvio Ollama

```

# 1. Installa Ollama (Mac/Linux)
curl -fsSL https://ollama.com/install.sh | sh

# 2. Scarica il modello
ollama pull llama3.2:3b

# 3. Avvia il server (si avvia automaticamente alle richieste API
)
ollama serve

```

6.2 Ordine di esecuzione

1. **L1:** apri `L1_LLM_data_quality_assessment.ipynb` e esegui tutte le celle. Richiede che gli scorecard DQ siano già in `4_dq_scorecards/` (generati da `data_quality_assessment_LOC`).
2. **L2:** apri `L2_Colab2_LLM_Missing_Outliers.ipynb` e esegui tutte le celle. Legge direttamente i CSV raw da `1_raw_data/`.
3. **L3:** apri `L3_Colab2_LLM_Cleaning.ipynb` e esegui tutte le celle. Richiede sia i CSV raw che i cleaned da `3_cleaned_data/`.

6.3 Output generati

Tutti gli output vengono salvati in `10_script_llm/outputs/`:

Script	File	Contenuto
L1	<code>dq_audit_{table}.md</code>	Audit report per ogni tabella
L1	<code>dq_generated_rules_{table}.md</code>	Regole di validità Python
L1	<code>stakeholder_summaries.md</code>	Sommari per management
L2	<code>L2_missing_interpretation.md</code>	Interpretazione MCAR/MAR/MNAR
L2	<code>L2_outlier_narrative.md</code>	Narrativa outlier
L2	<code>L2_imputation_code.py</code>	Codice imputazione generato
L3	<code>L3_category_mapping.json</code>	Mapping PT→EN finale
L3	<code>L3_cleaning_plan.json</code>	Piano cleaning strutturato
L3	<code>L3_audit_narrative.md</code>	Narrativa audit prodotti

Table 5: Output generati dalla pipeline LLM

7 Considerazioni Metodologiche

7.1 Limiti degli LLM in questo contesto

- **Non determinismo:** la stessa chiamata LLM può produrre output diversi tra esecuzioni successive. Per questo il codice generato viene sempre validato prima dell'esecuzione (`exec()` in sandbox).
- **Allucinazioni:** l'LLM può generare regole di validità plausibili ma errate (es. formato errato per `order_id`). Le regole generate devono essere revisionate manualmente prima di essere usate in produzione.
- **Dipendenza dal modello:** modelli più piccoli (l)
- **Dipendenza dal modello:** modelli più piccoli (llama3.2:3b) producono output meno precisi rispetto a GPT-4o o Claude Opus. Per produzione è consigliato usare un modello cloud.

7.2 Sicurezza del codice generato

Tutti i blocchi di codice Python generati dall'LLM vengono eseguiti in un ambiente sandbox isolato:

Listing 13: Esecuzione sicura del codice LLM

```
local_env = {}    # ambiente isolato -- non accede a variabili
                  globali
try:
    exec(code_str, {}, local_env)
    result = local_env.get('category_mapping', {})
except Exception as e:
```

```
print(f'Parsing LLM code fallito: {e}')  
result = {}
```

7.3 Conclusioni

Gli script L1, L2 e L3 dimostrano un approccio ibrido alla Data Quality: la pipeline quantitativa classica fornisce numeri oggettivi e riproducibili, mentre gli LLM aggiungono interpretazione domain-aware, generazione di codice e comunicazione verso stakeholder non tecnici. L'adattamento al dataset Olist rende l'intera pipeline eseguibile localmente senza dipendenze da Google Colab o da API chiave per il provider Ollama.